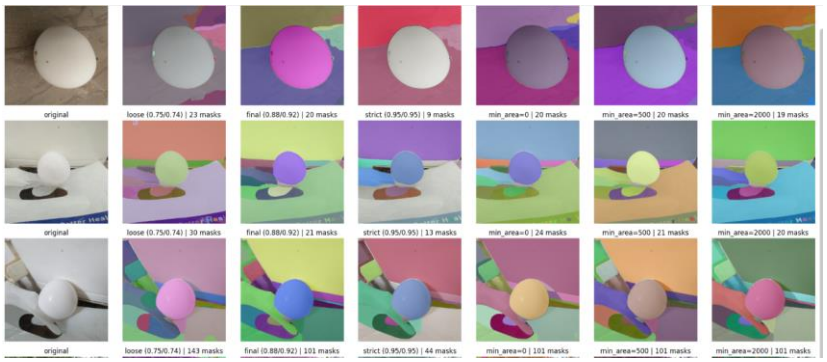
Task 1: Segmentation using a Pretrained Model (SAM)

Why SAM. The dataset has only images and no masks, so I could not train a model like U-Net, which I used in my lip segmentation project. A model like that needs labelled data. SAM (Segment Anything Model) is already pretrained on a very large dataset, so it can segment objects without training. I learned about it through articles and AI tools, and I used it in automatic mask mode on all 108 images.

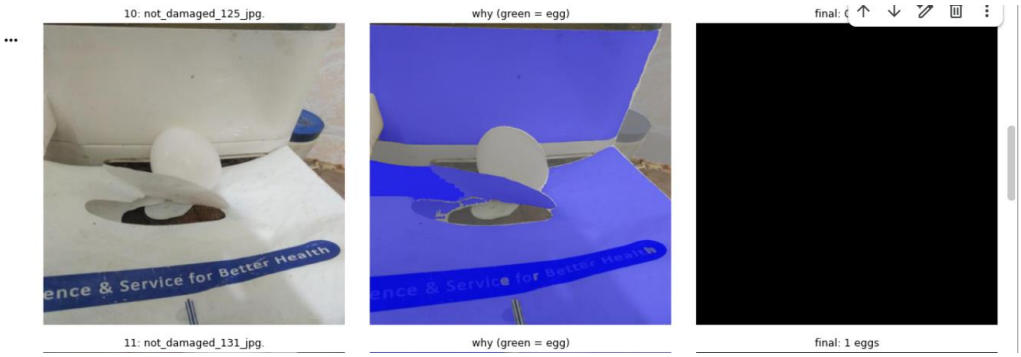
Problem with the raw output. SAM segments everything: eggs, background, shadows, hands, walls and books.

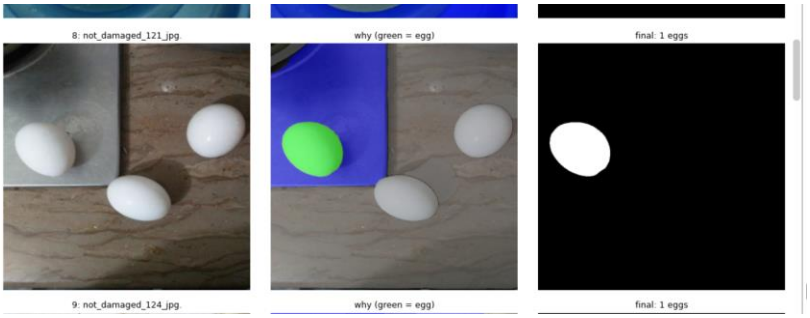


So I added filters to keep only the egg masks.

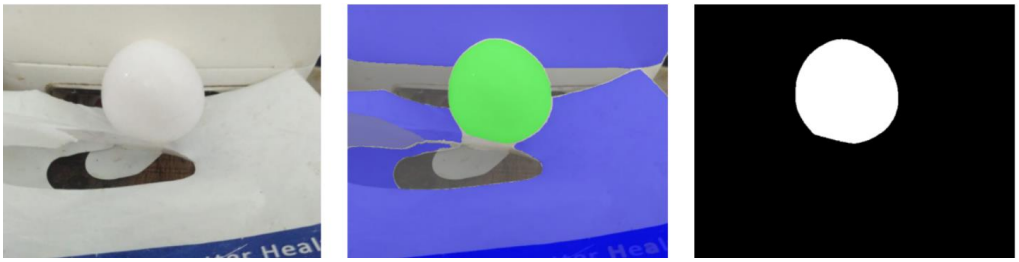
Filter 1: size. I keep only masks whose area is between 3% and 60% of the image area. Very small regions are noisy, and very large regions are background.

- With 0.02 to 0.4, some eggs were removed.





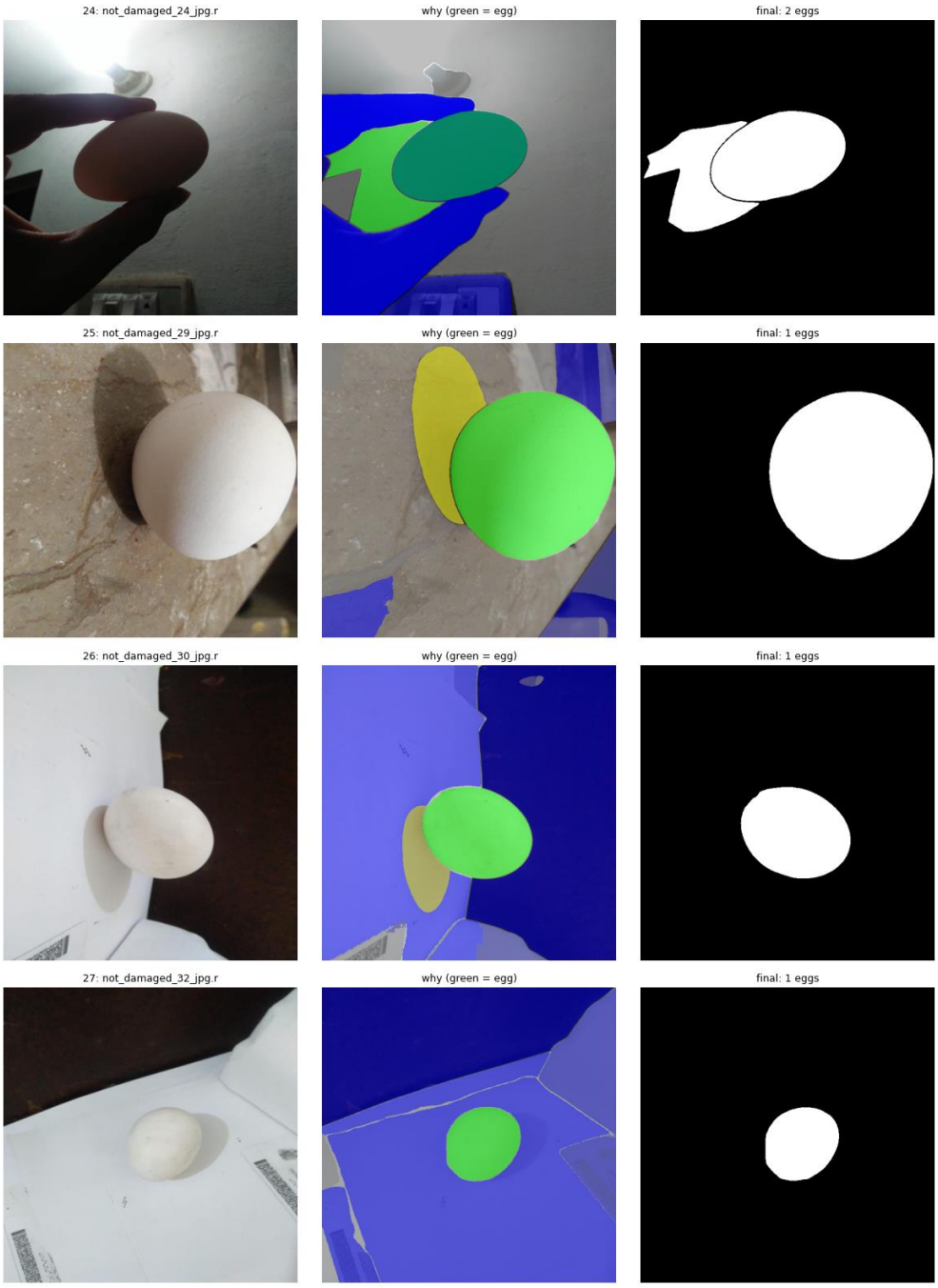
- with 0.03 to 0.6: the eggs were kept. I chose this.



Filter 2: border. I remove masks that touch the image edge. The hand, wall and book regions usually connect to the frame edge, so this removes them. Blue region removed

Filter 3: shadow. Some egg shadows were also segmented. A shadow is darker than the egg, so I remove masks whose mean brightness is below the image's mean brightness.yellow region removed

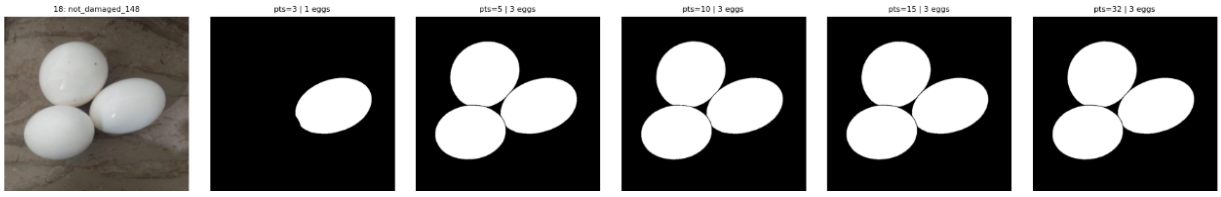
After all three filters the results were much cleaner.



Parameter choices

points_per_side = 15. This controls how many points SAM uses as prompts across the image. I compared 3, 5, 10, 15 and 32.

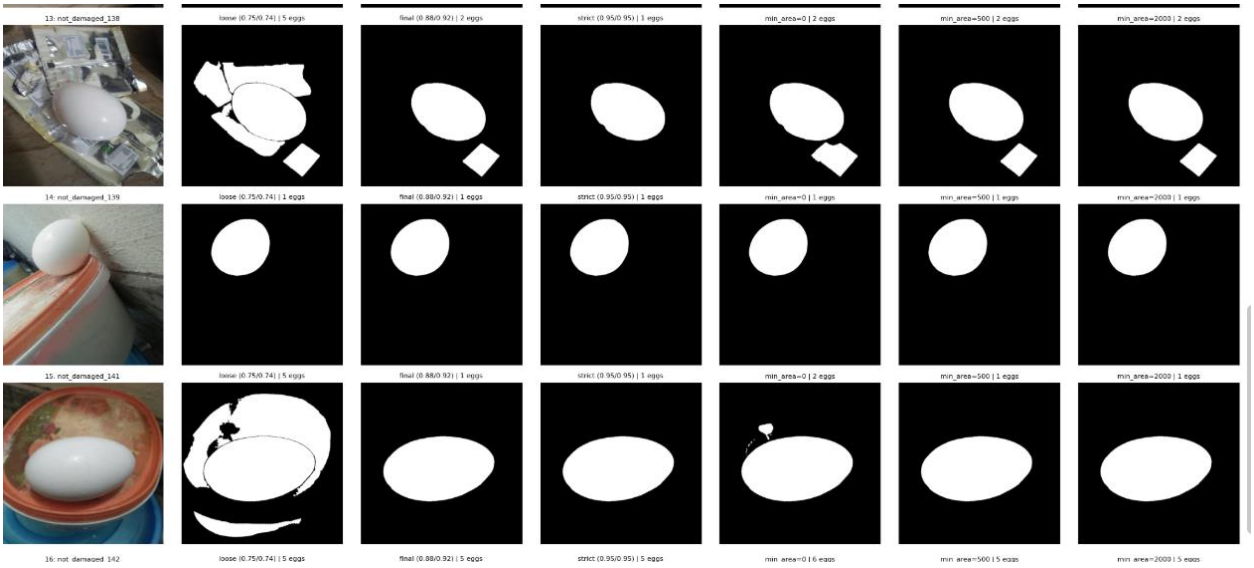
- At 3 and 5, some eggs were missed and extra regions appeared in some images.
- 10, 15 and 32 looked similar to me, so I picked 15 as a middle value between speed and coverage.



pred_iou_thresh = 0.88. This filters masks by SAM's own quality score. At 0.75 too many unnecessary regions were included, and at 0.95 small eggs were ignored.

stability_score_thresh = 0.92. Tested the same way. Lower values added unnecessary regions and 0.95 dropped small eggs.

min_mask_region_area = 500. I did not experiment much with this. On the sample images the results were the same, so I kept it.





Limitations

- After filtering, 19 of 108 images still had wrong masks.
- I found them by **manually comparing each SAM mask with its original image**. No metric was used.

- I moved these 19 images to a `bad_mask` folder and left them out of all IoU metrics.
- I did not measure the reasons below. They are my guesses from how the filters work and from what I saw in the images.

Likely causes

1. **Shadow on the egg:** the shadow filter removes any mask darker than the image average. A shaded egg can get removed the same way as a shadow.
2. **Bright white background:** white or shiny areas are brighter than average, so they pass the shadow filter. If they don't touch the border and are in the size range, nothing removes them.
3. **Egg touching the image edge:** the border filter removes every mask that touches the edge, so an egg cut by the frame is lost.

Dataset split

- **Split:** 108 images into dev (86) and eval (22), about 4:1.
- **Use:** dev is for building and tuning the algorithm. Eval is run once at the end with the final settings.
- **Method:** `train_test_split` (test_size = 0.2, random_state = 42), done on the filename before `.rf.`, so images with the same source name stay together. The split is saved in `split.csv`.
- **Leakage check:** I looked through the images by eye. They are separate photos on the same background, each from a different angle. I saw no duplicates, but I did not check for near-duplicates with different filenames.
- **Clean subsets:** I removed images with clearly wrong SAM masks (17 dev, 2 eval). The results use 69 clean dev and 20 clean eval images.
- **Order:** I split first and excluded bad masks after, so the exclusion did not affect the split.
- **Limitation:** all images share one background, so dev and eval look similar. Eval scores may be a little optimistic on a new background.

Task 2: Classical image processing

- I started with grayscale → blur → threshold → contours, because I already know this pipeline.
- I also chose to try **watershed**. I had never used it, so I wanted to learn how it works by applying it to this task.
- The goal is to find the eggs in each image and match the Task 1 SAM masks as closely as possible.

Algo0: fixed threshold

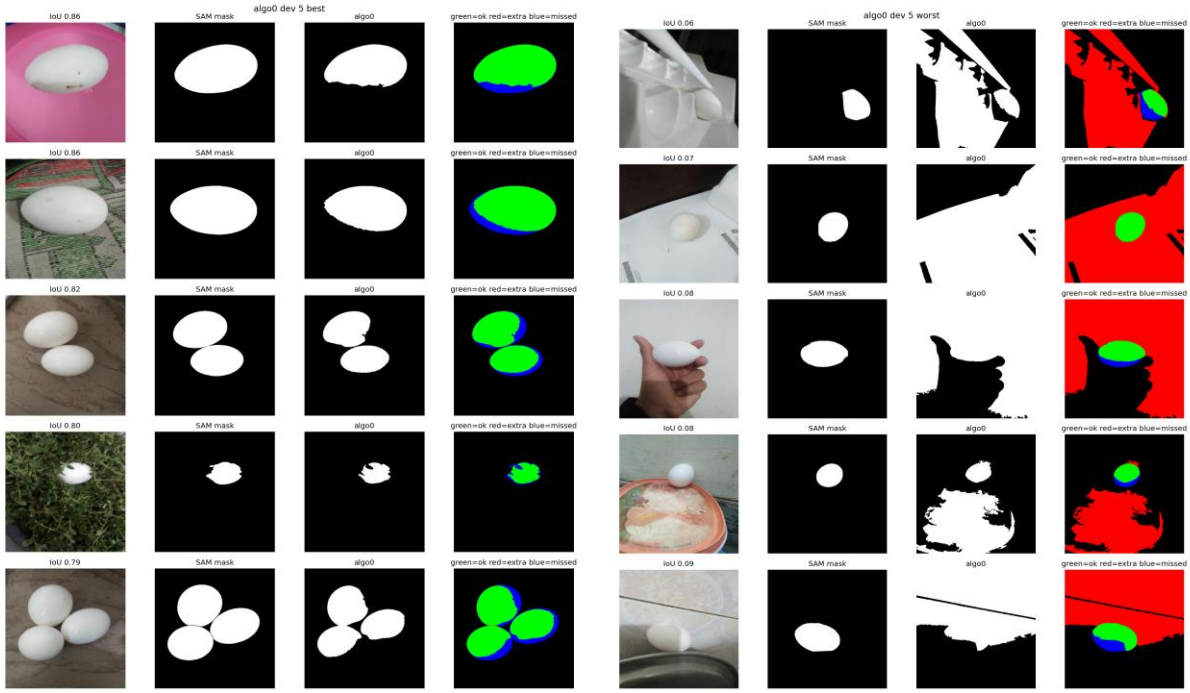
Idea. The egg is brighter than most backgrounds, so I convert the image to grayscale, blur it, and keep the pixels brighter than a fixed threshold. Then I keep only the large bright blobs and fill them to get the egg mask.

Parameter	Tested	Chosen	Reason
Threshold	90, 110, 127, 150, 170, 180, 190, 200	**170**	Best IoU (0.421). Lower values include the bright background, and higher values lose the shaded part of the egg
Blur kernel	3, 5, 7, 9, 15	**5**	No real change in IoU, so I kept the default
Blur sigma	0, 1, 2, 4	**0**	No effect, so I let OpenCV choose
Min area	100, 500, 2000, 3500, 5000	**5000**	Removes small noise blobs, and IoU rose as it increased (0.333 to 0.348)

Each parameter was varied from the baseline (threshold 127, blur 5, min area 500), and the combined final settings were then evaluated together.

Result

Set	Images	IoU	Dice	Precision	Recall
Dev	69	0.435	0.567	0.587	0.692
Eval	20	0.478	0.606	0.691	0.665



Eval was run once with the final settings (threshold 170, blur 5, min area 5000), and nothing was tuned on it. Eval is a bit higher than dev, but it has only 20 images, so I would not read that as the algorithm being better on new data.

Limitations

- **The mask does not fully cover the egg when part of it is in shadow.** In the best images, the shaded or darker part of the egg is missing from the mask. This is clear in the overlay images (the blue “missed” strip along the lower edge).
- **It fails when the background is also bright.** On a white tray, wall or tiles, the background passes the threshold too, so the mask covers the background. The 5 worst images had an IoU of 0.06 to 0.09.
- **One threshold for all images.** Lighting differs between images, so a single value of 170 can’t suit all of them.

Algo1: Otsu and adaptive threshold, with border blob removal

Idea. Algo0 uses one fixed number (170) for every image, but lighting differs between images, and on bright backgrounds the background also passes. So I changed the threshold method, and I also removed any white blob touching the image edge, because the background usually touches the edge and the egg usually doesn’t. This is the same idea as the border filter I used for the SAM masks. I tried two threshold methods:

- **Otsu:** picks the threshold for each image automatically from its brightness histogram.

- **Adaptive:** compares each pixel with the average brightness of its local neighbourhood, so it can handle uneven lighting.

Parameters and why

Parameter	Value	Reason
Blur	5x5, sigma 0	Same as algo0, so only the threshold method changes
Min area	5000	Same as algo0 (from the algo0 sweep)
Border removal	on	Removes blobs touching the edge (the background)
Block size (adaptive)	51	Local window size. Large enough to ignore small texture
C (adaptive)	-10	A pixel must be brighter than its surroundings by 10 to turn white

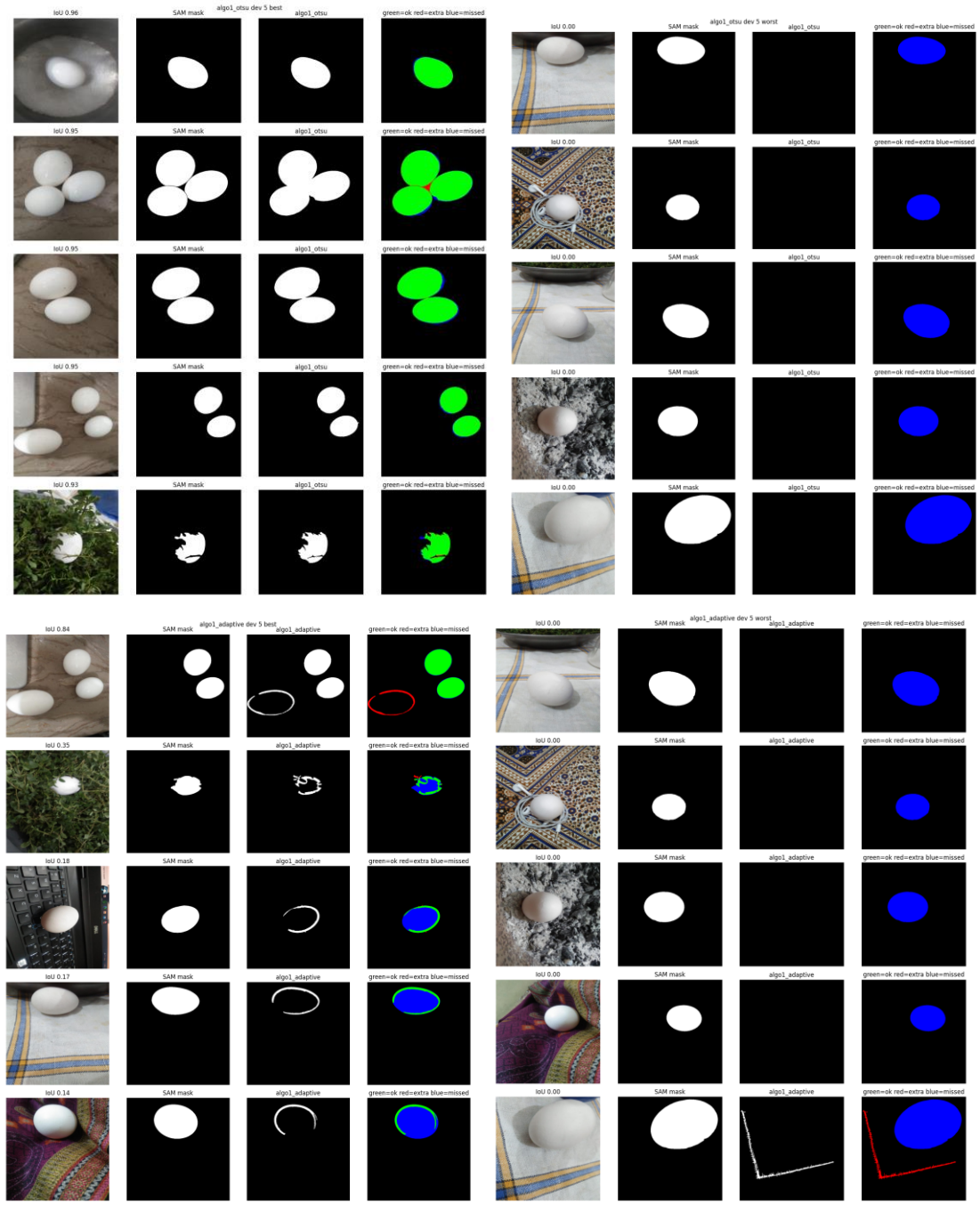
Otsu has no threshold parameter, because it chooses it. The adaptive values (51, -10) were starting values, and I did not tune them.

Result (dev set, 69 images)

Version	IoU	Dice	Precision	Recall
algo0 (reference)	0.435	0.567	0.587	0.692
Otsu + border	0.323	0.363	0.390	0.353
Adaptive + border	0.033	0.048	0.156	0.035

What I saw in the images

- **Otsu + border:** on simple eggs, the mask overlaps the egg better than algo0. But 39 of 69 images had an IoU near zero (almost nothing found), so the average dropped.
- **Adaptive + border:** it found almost nothing. In its best images, only a thin outline of the egg was marked, and the smaller details were missed.



Limitations (why they failed)

Otsu

- 39 of 69 images had an almost empty mask.

- A likely reason is the border rule. In algo2 I saw that when the egg and the background join into one blob touching the edge, the rule removes the whole blob, including the egg. I did not test Otsu without border removal, so I can't be sure this explains all 39.
- Otsu also failed on some simple eggs, and I did not find out why.

Adaptive

- It found almost nothing. In its best images only a thin outline of the egg was marked.
- A likely reason is that the inside of a large egg has about the same brightness as the area around it, so only the edge counts as brighter. I did not test this.

Both

- Both use brightness only, so they can't separate a white egg from a white background.

Decision

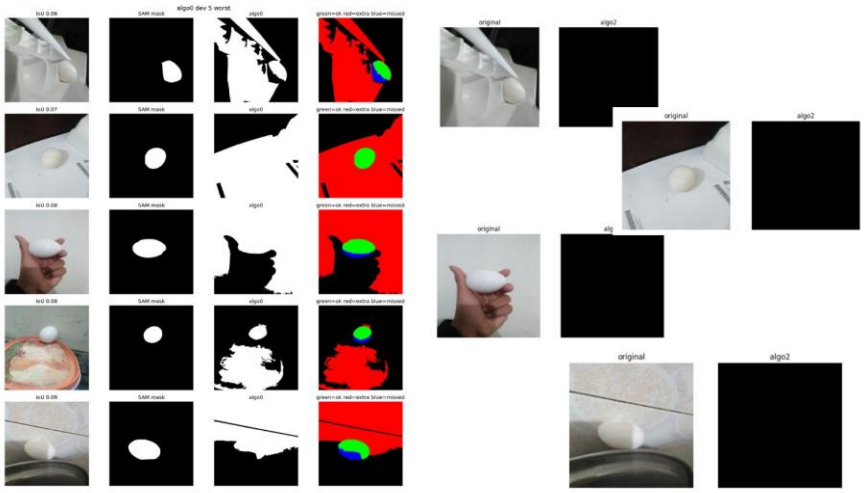
- Both were discarded, because both had a lower IoU than algo0

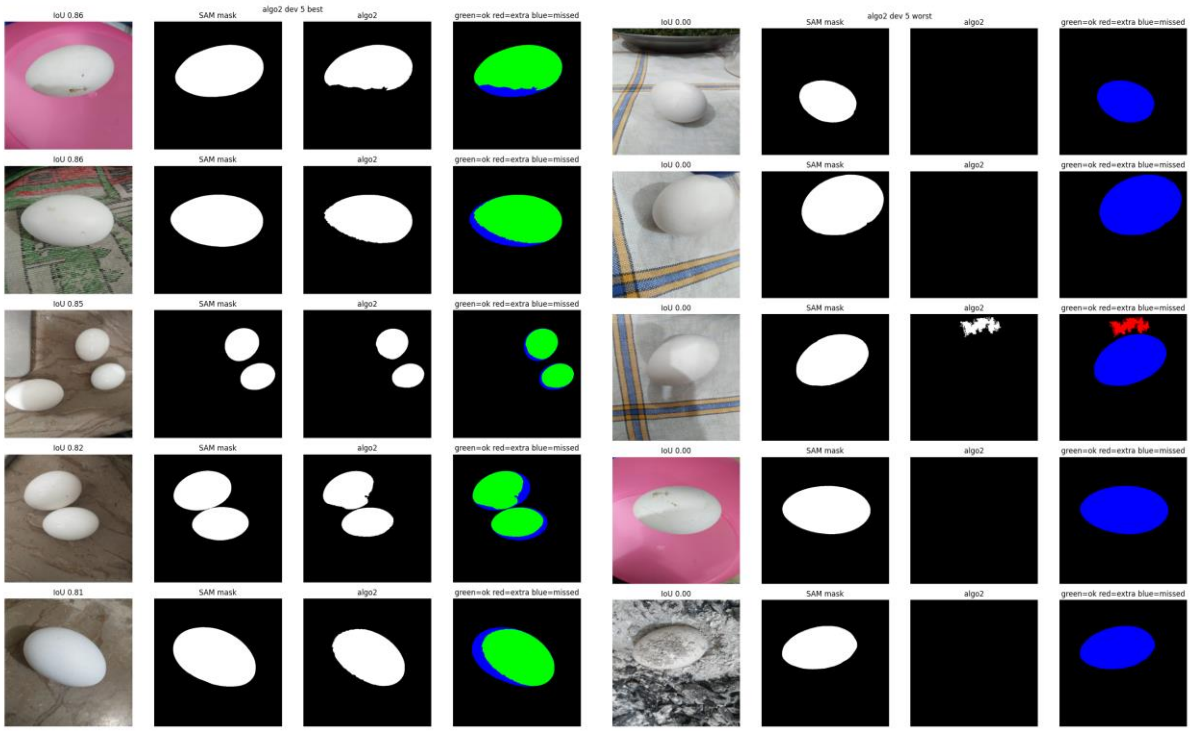
Algo2

Idea. same as algo0, plus one rule: remove any white blob that touches the image edge. The background usually touches the edge and the egg usually doesn't.

Parameters. threshold 170, blur 5, min area 5000 (from the algo0 sweep), border removal on.

Result (dev, 69 images): IoU 0.430, Dice 0.519, precision 0.643, recall 0.448, 19 empty masks. Algo0 had IoU 0.435, Dice 0.567, precision 0.587, recall 0.692.





What happened

- I compared the worst algo0 images with their algo2 results. In these images algo0 had marked the background as egg.
- After adding the border rule, the whole mask was removed and the result was blank. The border rule did not solve these cases. It only turned a wrong mask into an empty one.
- A likely reason is that the egg and the background were joined into one blob touching the edge, so the rule removed the whole blob, including the egg.

- Algo2 gave 19 empty masks out of the 69 clean dev images. I only checked a few of them, so I can't say they all have this cause.
- Precision went up and recall went down. IoU stayed almost the same (0.435 → 0.430), and Dice dropped (0.567 → 0.519).

Decision: not chosen. Algo0 is equal or better on IoU, Dice and recall.

Limitations:

- The border rule deletes egg-and-background blobs that are merged.
- The shadow problem remains.
- Both come from using brightness only.

Task 4: Video tracking (DeepSORT)

Note: I could not download the video from YouTube with yt-dlp, because it showed "video is not available". I asked Manjunath sir for a Drive link, and I ran the tracker on the video.

Idea. Run algo0 on every frame, turn each white blob into a bounding box, and give the boxes to DeepSORT, which assigns an ID to each egg and keeps it across frames.

How it works

- **Detection:** algo0 gives a mask, and each white blob becomes a box. I used `min_area = 500` here instead of 5000, because 5000 detected almost no eggs in the video (eggs look smaller in the frames). The other settings are the same. The Task 2 and 3 results still use 5000.
- **Tracking:** DeepSORT gives each box an ID and keeps it across frames. It uses motion (a Kalman filter predicts where the egg goes next) and appearance (image features compared by cosine distance).
- **Drawing:** only confirmed tracks seen in the current frame are drawn, with the box and ID.
- **Parameters:** `max_age = 30`, `n_init = 3`, `max_cosine_distance = 0.3`. I also tried `max_age = 50` and saw no clear difference by eye, so I kept these values.

Parameters

Parameter	Value	Meaning
max_age	30	A lost track is kept for 30 frames before deleting
n_init	3	A new track must be seen in 3 frames before it's confirmed
max_cosine_distance	0.3	How different two appearances can be and still match

VIDEO: **output.mp4**

What I saw in the video

- Unwanted white regions (bright background, other objects) also get boxes and IDs, because algo0 treats any bright blob as an egg.
- When eggs overlap or are hidden behind something, the ID can change after they reappear.

Why the IDs switch (likely causes, not tested)

- The eggs look almost identical, so the appearance feature probably can't tell them apart.
- DeepSORT then depends mostly on motion, and motion prediction fails after an occlusion.

Limitations and next steps

- **The tracker depends on the detector: wrong boxes come from algo0, not from DeepSORT.**
- **Next, I would filter boxes by egg-like size and shape, and try a simpler tracker.**

Use of AI tools. I used AI tools (Claude) and articles and blogs to learn how SAM works. AI helped write the SAM mask-generation code, the code that computes the result CSVs, and the code that saves the figures. The design of the filters, the experiments, the parameter choices, the comparison between algorithms and the decision to keep algo0 are my own work, and I can explain all of the code.

Conclusion

I completed all four tasks: SAM masks, a classical algorithm (algo0 chosen), the evaluation, and video tracking.

- The final method is a brightness threshold (algo0). It works on simple eggs but fails on shadows and bright backgrounds.
- I tried different thresholds, Otsu, adaptive threshold and border removal. None beat algo0.
- If I had more time, I would like to try other ideas for shadows and bright backgrounds.